Nurse
Althea

PARTNER TECHNICAL BRIEFING

Nurse Althea

A multi-school, AI-assisted nursing education platform with a simulated nurse conversation, a realistic EMR workspace, and structured case evaluation.

PREPARED	AUDIENCE	SOURCE BASIS
30 July 2026	**Partner technical team**	**Repository HEAD + migrations**

Executive takeaway. Nurse Althea combines deterministic case configuration and persistence with AI-generated conversational and clinical-simulation content. The core product is a browser-based React application; Supabase supplies identity, Postgres, storage, realtime, and server-side Edge Functions.

This brief describes the implementation visible in the repository at the date above. Exact production deployment status, Netlify site identity, Supabase region, retention policy, uptime, and compliance attestations should be confirmed with the operator before being represented as contractual commitments.

1. Product and user experience

Nurse Althea is an EMR simulation for nursing education. A student receives one or more assigned rooms/cases, interacts with a nurse persona through chat, reviews patient information in an EHR-style interface, places orders, reviews generated results, writes a progress note, and completes the case. Faculty and administrators configure cases, manage schools and users, review timelines, and inspect automated feedback.

Student

Practice

Chat with the nurse, inspect the patient chart, order labs/ medications/ imaging, review vitals and results, and submit a progress note.

Faculty

Teach

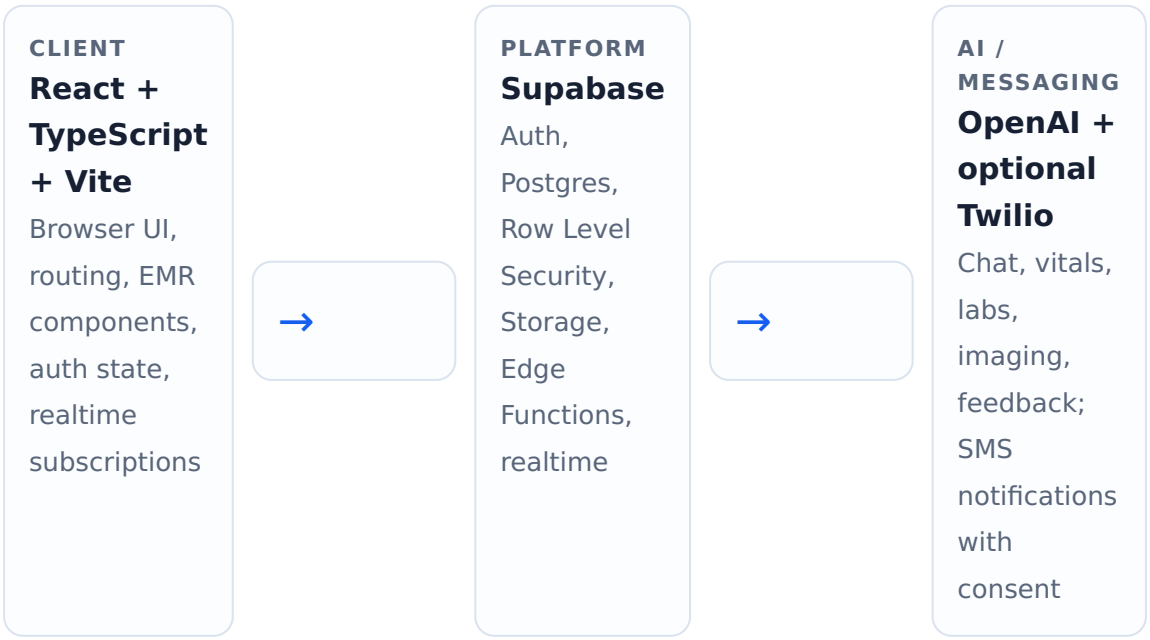
Author rooms and case context, assign cases to students or cohorts, schedule availability windows, and review performance.

Platform admin

Operate

Manage schools, roles, bulk users, room access, assignments, feedback regeneration, exports, and cross-school reporting.

2. System architecture



Layer	Current implementation	Partner implication
Web app	Vite production build, React 18, TypeScript, Tailwind, React Router, Zustand.	Static-hosting friendly; SPA fallback is configured through `netlify.toml`.
Data and auth	Supabase project reference `lvpbwtfvairspufrashl`; `@supabase/supabase-js` client.	Managed Postgres and Supabase Auth are the system of record. The public client uses an anon key; privileged keys stay in functions/server scripts.
Server logic	Deno-based Supabase Edge Functions.	AI provider credentials and service-role access are intended to remain server-side.
External messaging	Twilio integration for assignment-effective SMS, gated by phone number and SMS consent.	Requires Twilio account secrets and a configured sending number; SMS is optional to the learning flow.

3. How a case works end to end

1. An administrator creates a room/case with nurse persona, clinical/EMR context, objectives, difficulty, expected diagnosis/treatment, orders configuration, optional PDFs, and a linked patient.
2. The case is assigned to a student. Assignment windows can activate cases at scheduled/effective times, with optional SMS notification when the assignment becomes effective.
3. The browser loads the student assignment and subscribes to assignment-scoped chat updates. The initial nurse prompt is derived from the room role, style, and nurse context.
4. The student sends a message. The client invokes the `chat` Edge Function; the function calls OpenAI and persists/returns the assistant response.
5. Orders, notes, vitals, labs, and imaging are written to the EMR tables with scope metadata. STAT/manual generation invokes the relevant Edge Function; local generators can act as fallbacks for some lab/vital paths.
6. Completion saves status, timestamp, and the student progress note, then triggers evaluation. The evaluator builds a chronological timeline of conversation and clinical actions.
7. Feedback is stored against the assignment, including communication and medical-decision-making scores, detailed breakdowns, learning objectives, case difficulty, and legacy feedback fields.

4. Data model and isolation model

The model is relational Postgres with UUIDs for users, patients, assignments, and clinical artifacts; rooms are integer-keyed. Records carry school ownership and, for simulated clinical activity, assignment/room scope.

Entity	Purpose / important fields	Relationships
`schools`	Institution/tenant record; name, slug, timezone.	Parent for profiles, rooms, patients, assignments, and clinical artifacts.

`profiles`	User profile; role is student, school_admin, super_admin, or test_user; school affiliation, email, phone, SMS consent.	Backed by Supabase Auth user identity.
`rooms`	Case container; room number/role/style, nurse and EMR context, diagnosis/ treatment, goals, difficulty, objective, progress-note prompt, completion hints, orders config, PDF, continuity.	May link to a patient, specialty, school access, and prior room.
`patients`	Simulated patient demographics, MRN, service, attending, allergies, code status, room, custom overview content.	One patient can have clinical artifacts and be attached to a room.
`student_room_assignments`	Student-to-room attempt; status, scheduling window, completion, progress note, grade, feedback status, scores, breakdown JSON.	Joins one student to one room; primary unit of student activity and evaluation.
Clinical artifacts	`clinical_notes`, `medical_orders`, `lab_results`, `vital_signs`, `imaging_studies`.	Patient-linked; can be baseline, room-scoped, or assignment-scoped. Soft deletion uses `deleted_at` where implemented.
`chat_messages`	Chronological nurse/student messages keyed by assignment.	Realtime subscription and evaluation timeline source.

> **Scope semantics.** Baseline data is shared case setup; room data is the configured scenario layer; assignment data is a student's isolated attempt. The EMR API and evaluation timeline filter artifacts by these

scopes so one student's generated activity does not become another student's chart.

5. AI services and guardrails

Conversational nurse

`chat` receives an assignment ID and message history, then calls OpenAI model `gpt-5-mini`. Room instructions tell the assistant to stay in character, be concise, avoid suggesting treatments, and ask for physician orders when appropriate.

Clinical simulation generators

`lab-results`, `vitals-generator`, and `imaging-results` receive patient, room, assignment, chart, order, and case context. They return structured JSON that the app persists in the corresponding EMR tables.

Evaluation

`generate-feedback` assembles chat, orders, labs, vitals, imaging, completion, and the progress note into a chronological evaluation timeline. It scores Communication and Medical Decision Making on 0–5 scales with rubric breakdowns and optional case leniency.

Operational functions

Bulk user creation, effective-assignment processing, feedback reruns/exports, superadmin reporting, and SMS support administrative workflows.

Important product boundary. AI outputs are simulated educational content, not clinical advice, diagnostic output, or a production medical record. Partner-facing messaging should describe Nurse Althea as a training/simulation platform.

6. Security, privacy, and diligence notes

Controls visible in the code

- Supabase Auth supplies authenticated sessions; application roles and school membership drive UI and many database policies.
- Privileged operations use the Supabase service-role key inside Edge Functions rather than the browser bundle.
- Chat, export, reporting, and feedback paths use assignment IDs; reporting/export functions explicitly validate the caller in code.
- Storage-backed room PDFs and imaging assets are accessed through signed URLs in the client flow.
- SMS requires an explicit consent flag and a phone number.
- Clinical records support soft deletion and timestamps; database changes are versioned as migrations.

Items to verify or harden before production partner commitments

- Some legacy EMR migrations use broad “authenticated users” read/write policies, while later migrations add more specific role/school/assignment policies. Confirm the effective remote policy set and tighten it to least privilege before storing real PHI.
- Several functions have `verify_jwt: false` in local function config. Their endpoint-level authorization should be standardized, and public invocation should be restricted where appropriate.
- Confirm the deployed function versions match this repository, especially the current model, prompts, auth checks, and migrations.
- Confirm production hosting provider/site, Supabase region, backups/PITR, retention/deletion, audit logging, incident response, encryption statements, and any HIPAA/BAA posture separately; they are not established by this repository alone.
- Review repository/CI secret hygiene and rotate any credentials that may have appeared in git remotes, logs, or committed configuration. Never distribute service-role, OpenAI, Twilio, or source-control credentials in partner materials.

7. Deployment and operations

Activity	Current workflow in repo	Production question for partner
Frontend	`npm run build` generates `dist/`; SPA rewrite sends all routes to `index.html`.	Which Netlify site/account owns production, and how are preview/prod environments separated?
Database	Supabase SQL migrations under `supabase/migrations`.	Who approves migrations, and what rollback/backup process is used?
Functions	`npx supabase functions deploy <name>`; secrets are set with Supabase secrets.	Are functions deployed from CI with environment-specific secrets and logs/alerts?
Quality	Vitest tests are present; build/lint commands are defined. Manual EMR and assignment flows remain important.	What release gates and test accounts are used for partner environments?

8. Technical FAQ

Where is Nurse Althea hosted?

The application is built as a static Vite SPA and includes Netlify SPA routing configuration. The backend is Supabase at the project URL associated with project reference `lvpbwtfvairspufrashl`. The exact production web site, Supabase region, and active deployment should be confirmed by the operator.

Does the browser call OpenAI directly?

The intended architecture is server-mediated: the browser calls Supabase Edge Functions, and those functions call OpenAI using a server-side secret. The public browser configuration contains only the Supabase URL and anon key.

What does the AI see?

Depending on the operation, prompts can include simulated patient demographics, allergies/code status, room and assignment identifiers, nurse/EMR context, objectives, notes, prior vitals/labs, orders, progress-note context, and completion hints. The platform should use synthetic data unless a reviewed privacy/compliance design says otherwise.

Can the AI place orders or change the database by itself?

The model returns text or structured generated results; application code persists chat and clinical artifacts. The student explicitly places orders through the UI. The model is not an autonomous clinical actor.

How are students isolated?

Assignments identify a student's attempt. Clinical records include `assignment_id`, `room_id`, and `override_scope`; reads combine only relevant baseline, room lineage, and current assignment rows. Effective RLS must still be verified in the deployed database.

What is a room?

A room is a configurable case/scenario, not merely a physical bed. It contains persona, case metadata, patient linkage, learning objectives,

documents, orders configuration, and optional continuity from a previous room.

How is feedback generated?

After case completion, the evaluator reads the full chronological activity timeline and calls OpenAI with a fixed rubric. It stores structured scores, feedback, learning objectives, and breakdown JSON on the assignment.

What happens if generation fails?

The UI surfaces errors; lab and vital paths retain local fallback generators in the client architecture. Feedback tracks processing/completed/failed state and an error field, and administrators can rerun assessments.

Is there realtime?

Yes. The chat interface subscribes to assignment chat changes and updates the timeline as messages and clinical events are persisted.

Can a school use its own content?

The multi-school model supports school affiliation, school admins, room access by school or all schools, case blueprints, assignments, and bulk user creation. Confirm the desired content ownership and cross-school sharing policy during integration planning.

Does it integrate with a real EHR?

The repository shows an internal simulated EMR, not a production EHR integration or FHIR API. A real EHR, SSO, LMS, roster, or gradebook integration would be a separate scope.

Is Nurse Althea HIPAA compliant?

The repository alone cannot establish compliance. Treat it as a synthetic-data educational simulator until hosting, access control, audit, retention, encryption, incident response, and contractual requirements have been reviewed.

How are documents and images handled?

Room PDFs and imaging assets are stored through Supabase Storage and served via signed URLs in the application flow. Bucket policies and expiry durations should be confirmed in the live project.

What is the expected scale?

No capacity or load-test numbers are defined in the repository. Supabase and OpenAI limits, concurrent student sessions, prompt/token volume, function timeouts, and storage growth should be load-tested for the partner’s cohort size.

What would an integration project need?

Agree on identity/SSO, roster and school mapping, case/content ownership, assignment lifecycle, reporting/export fields, privacy boundaries, environment separation, support ownership, and an acceptance test using synthetic cases.

---

9. Suggested partner discussion checklist

Technical

Confirm deployment topology, region, environment separation, SSO/LMS/roster needs, rate limits, observability, backups, and integration contracts.

Clinical and academic

Agree on case authoring, rubric ownership, faculty override/ review, content update process, safety language, and what is explicitly simulation-only.

Privacy and security

Define synthetic versus real data, PHI boundaries, retention, deletion, audit events, access reviews, incident response, vendor agreements, and least-privilege policies.

Commercial / rollout

Define pilot cohort, schools, success metrics, support SLA, training, migration/import plan, release cadence, and ownership after handoff.